AdStudio Technical FAQs

Java 25 LTS — Why Not 17 or 21?

Java 17 LTS

- **Minimum required by Spring Boot 3.x — it was the baseline, not the ideal**
- **No virtual threads — concurrency handled via traditional thread pools (expensive under load)**
- **No record patterns, no pattern matching for switch — more verbose code**
- **Oracle JDK 17 updates after September 2024 moved to a paid subscription model — free use window already closed Oracle**
- **Verdict: Too old for a greenfield project in 2026. Security and licensing risks.**

Java 21 LTS

- **Virtual threads introduced (Project Loom) but still considered preview/early in ecosystem**
- **Pattern matching for switch finalized**
- **Records and sealed classes stable**
- **Java 21 updates will get patches until September 2026 — only a few months away — you'd be upgrading almost immediately after launch Charlie Arehart**
- **Verdict: Solid, but short remaining free support window makes it a poor choice for a new project today**

Java 25 LTS — Why This One

- **Java 25 is the latest LTS released September 2025, with support until September 2033, and is the recommended Java version for greenfield development — Spring Boot 4.0 and Spring Framework 7.x work best with Java 25 Mac Install Guide**
- **Virtual threads fully mature and production-battle-tested — directly useful for AdStudio's 8,000+ concurrent user requirement**
- **Longest free support runway — no forced upgrade for 7+ years**
- **JDK 25 binaries are free to use in production under Oracle's No-Fee Terms and Conditions Oracle**

How Java 25 features will be used in AdStudio

- **Virtual threads — AdStudio's Publisher Portal and Media Planner Console will have many simultaneous users hitting delivery record and IO endpoints during campaign launches. Virtual threads handle thousands of lightweight concurrent requests without the overhead of a traditional thread pool**
- **Records — Used for DTOs (request/response objects). Instead of verbose POJO classes with getters/setters, a CampaignBriefResponse becomes a concise record — cleaner and immutable by default**
- **Pattern matching for switch — Useful in the StatusTransitionValidator where different entity types (CampaignBrief, InsertionOrder, CreativeAsset) each have their own allowed state transitions**
- **Sealed classes — Can model the strict status enums (e.g. InsertionOrder statuses: Sent → Confirmed/Rejected → Delivered/Disputed) as sealed hierarchies, enforced at compile time**

Spring Boot 4.0.6 — Why Not 3.x?

Spring Boot 3.5 (latest 3.x)

- **Still receives patches but Spring Boot 3.5 support ends June 30, 2026 — effectively EOL within weeks Eosl**
- **Built on Spring Framework 6, not 7 — misses API versioning, JSpecify, and modular JARs**
- **Works well with Java 17–21 but not optimized for Java 25**
- **Verdict: Dying branch. Starting a new project on it today means an immediate major upgrade**

Spring Boot 3.3 / 3.4

- **Both already EOL as of December 2025**
- **No new security patches**
- **Verdict: Not acceptable for a new production system**

Spring Boot 4.0.6 — Why This One

- **Built on Spring Framework 7, requires Java 17 minimum, adds first-class Java 25 support, introduces stable API versioning for HTTP endpoints, modularizes the Spring Boot codebase into smaller JARs, and replaces older null-safety annotations with JSpecify HeroDevs**

- **Active support through December 31, 2026 with Spring Boot 4.1 already in milestone — clear upgrade path Eosl**
- **Spring Security 7 (bundled) — modernized JWT and OAuth2 handling out of the box**

How Spring Boot 4.0.6 features will be used in AdStudio

- **Stable API versioning — AdStudio's PRD mentions a Phase 2 (DSP/SSP integrations, ad server feeds). Versioned endpoints (/api/v1/campaigns, /api/v2/campaigns) mean Phase 2 can introduce breaking changes without disrupting Phase 1 consumers**
- **Modular JARs — Smaller deployment artifacts. Relevant when deploying each of the 8 modules independently in a future microservices evolution**
- **JSpecify null-safety — All DTOs and service method signatures get compile-time null checks — reduces NullPointerException bugs in the billing and reconciliation logic where null VarianceAmount or AgencyCommission values would cause silent calculation errors**
- **Spring Security 7 — JWT implementation for the 6-role RBAC system is cleaner and requires less custom configuration than 3.x equivalents**
- **Virtual thread integration — Spring Boot 4.x is designed to automatically use Java 25 virtual threads for the embedded Tomcat request handling — zero extra config needed for the concurrency gains**

MySQL 8.4 LTS — Why Not 8.0 or 9.7?

MySQL 8.0

- **MySQL 8.0 reached End of Life in April 2026 with version 8.0.46 — no new security patches MySQL**
- **Verdict: Hard no. Cannot build a new production system on an EOL database**

MySQL 9.7 LTS

- **MySQL 9.7.0 LTS was just released April 21, 2026 — it establishes the next long-term support release line and introduces the Hypergraph optimizer, Dynamic Data Masking, and OpenID authentication Oracle**
- **Brand new — Flyway, Hibernate, and Spring Data JPA compatibility needs validation time**

- **No production hardening yet — first bugs surface in the weeks after a major release**
- **Verdict: Too fresh for a 15-day sprint with a 5-person team. The risk isn't worth it**

MySQL 8.4 LTS — Why This One

- **Direct successor to 8.0 — well-understood migration path, zero surprises**
- **Full, validated compatibility with Hibernate 6.x, Spring Data JPA, and Flyway**
- **Improved query optimizer over 8.0 — better execution plans for AdStudio's multi-join analytics queries**
- **Native JSON column improvements — useful for ChannelMix and ChannelPreferences fields in MediaPlan and CampaignBrief**
- **Production-hardened LTS — real-world battle testing across millions of deployments**

How MySQL 8.4 features will be used in AdStudio

- **Improved optimizer — The analytics layer (GET /api/reports/campaign/{briefId}) joins DeliveryRecord, MediaLineItem, and InsertionOrder across potentially millions of rows. The 8.4 optimizer generates better execution plans for these aggregations**
- **Native JSON columns — ChannelMix (stored as TEXT in the schema) could be upgraded to a proper JSON column in 8.4, enabling querying by individual channel values without full-text scanning**
- **Better replication — When AdStudio scales to a read replica setup (high-traffic delivery tracking reads vs writes), 8.4's improved replication observability makes that easier to manage**
- **Invisible indexes — During the Day 14–15 performance testing phase, indexes can be toggled invisible to measure their impact without dropping them — safer index tuning**

React 19.2.7 — Why Not 17 or 18?

React 17

- **No concurrent rendering — UI updates are synchronous, meaning large data loads (pacing dashboards, billing calendars) would block the UI thread**
- **No automatic batching of state updates**

- **Essentially unmaintained — only critical security backports**
- **Verdict: Completely unsuitable for a data-heavy multi-portal application**

React 18

- **Introduced concurrent rendering and automatic batching — a significant improvement**
- **Stable and widely used — massive library compatibility**
- **But missing the Actions API, which React 19 finalizes**
- **All previous major releases receive only critical security fixes — React 18 is now in maintenance mode End of Life Date**
- **Verdict: Safe but no longer the recommended track for new projects**

React 19.2.7 — Why This One

- **React 19.2.6 was released May 6, 2026 as the latest stable release, with 19.2.7 following June 1 — actively maintained Wikipedia**
- **Actions API finalized — purpose-built for async operations that match AdStudio's workflows exactly**
- **Improved form state management natively — no need for heavy third-party form libraries like React Hook Form for every portal form**

How React 19 features will be used in AdStudio

- **Actions API — Every portal has async submit flows: campaign brief submission, IO confirmation, creative approval decisions, invoice issuance. React 19 Actions handle the pending/error/success states of these operations natively, replacing verbose useState + useEffect chains**
- **useOptimistic hook — In the Publisher Portal, when a publisher confirms an IO, the status can update optimistically in the UI instantly while the API call completes in the background — better perceived performance**
- **use() hook for data fetching — Cleaner data loading for dashboard components like the pacing alert board and billing calendar, without needing Redux or heavy state management libraries**
- **Improved error boundaries — AdStudio has 6 portals with independent data flows. If the Finance Dashboard API call fails, React 19's improved error boundaries prevent it from crashing the rest of the application**

- **Ref as prop — forwardRef wrappers are no longer needed for custom form components (used heavily across all 6 portals' input forms) — cleaner, less boilerplate code**